

Entendendo o `__init__`

O que ele é, o que acontece por dentro, e os erros mais comuns na primeira vez que ele aparece.

RESUMO, SE VOCÊ SÓ TEM 1 MINUTO

`__init__` é um método especial que roda sozinho, automaticamente, toda vez que você cria um objeto novo. Ele existe pra preencher os atributos do objeto na hora da criação, em vez de você ter que preencher campo por campo depois, como no Dia 1. Ele sempre recebe `self` como primeiro parâmetro — `self` é o próprio objeto sendo criado. O resto deste texto explica isso com calma, passo a passo, com os erros mais comuns já resolvidos.

O QUE TEM AQUI

1. Relembrando o Dia 1

2. O que é um construtor

3. Anatomia do `__init__`, pedaço por pedaço

4. O que acontece por dentro, passo a passo

5. `self` em profundidade

6. Erros comuns (e como consertar)

7. Perguntas frequentes

8. Dia 1 vs Dia 2, lado a lado

9. Checklist mental

10. Glossário rápido

1. Relembrando o Dia 1

No primeiro encontro, a gente criou a classe `Personagem` e preencheu os campos de cada objeto na mão, um de cada vez:

```
class Personagem:
    nome = ""
    vida = 0
    ataque = 0

thoric = Personagem()
thoric.nome = "Thoric"
thoric.vida = 100
thoric.ataque = 15
```

Isso funciona. Mas repara no padrão: toda vez que um personagem novo nasce, são no mínimo quatro linhas — uma pra criar o objeto vazio, e uma pra cada atributo. Com dois personagens já são oito linhas fazendo exatamente a mesma coisa, só com valores diferentes. Com dez personagens, isso vira o mesmo tipo de bagunça repetitiva que motivou a gente a criar classes em primeiro lugar.

O `__init__` existe pra resolver exatamente esse problema.

2. O que é um construtor

Construtor é o nome que se dá, em várias linguagens de programação (não só Python), pro pedaço de código que roda automaticamente quando um objeto é criado. Em Python, esse construtor tem um nome fixo: `__init__` — de "initialize", inicializar.

Pensa em como funciona a criação de personagem em praticamente qualquer RPG: você não começa com um personagem em branco pra depois, tela por tela, escolher o nome, depois a classe, depois os atributos. Existe uma *tela de criação* onde você entra com tudo de uma vez, aperta confirmar, e o personagem já nasce pronto, com todos os campos preenchidos ao mesmo tempo. O `__init__` é essa tela de criação — só que escrita em código.

Em uma frase: sem `__init__`, você cria o objeto e depois o preenche. Com `__init__`, o objeto já nasce preenchido.

3. Anatomia do `__init__`, pedaço por pedaço

Aqui está a mesma classe `Personagem`, agora com construtor:

```
class Personagem:
    def __init__(self, nome, vida, ataque):
        self.nome = nome
        self.vida = vida
        self.ataque = ataque
```

Vamos abrir essa linha por linha, sem pular nada:

`def __init__(self, nome, vida, ataque):`:

- `def` — é um método, como qualquer outro dentro da classe (igual `mostrar_status` ou `receber_dano` que vocês já viram).
- `__init__` — esse nome não é escolha sua. É um nome reservado: sempre que Python cria um objeto novo dessa classe, ele procura por um método chamado exatamente `__init__` e roda ele sozinho. Os dois underlines antes e dois depois (`__`) marcam que é um nome especial, reconhecido pelo próprio Python — não é uma convenção estética, é assim que o Python identifica esse método como o construtor.
- `self` — sempre o primeiro parâmetro de qualquer método dentro de uma classe. Representa o próprio objeto que está sendo criado. Mais sobre isso na seção 5.
- `nome, vida, ataque` — parâmetros normais, exatamente como em qualquer função. São os valores que *você* vai fornecer na hora de criar o personagem.

`self.nome = nome`

Essa linha costuma confundir na primeira leitura, porque `nome` aparece dos dois lados. São duas coisas diferentes:

- O `nome` da **direita** é o parâmetro — o valor que chegou de fora, quando alguém chamou `Personagem("Thoric", 100, 15)`.
- O `self.nome` da **esquerda** é o atributo sendo criado *dentro do objeto*. É esse `self.nome` que vai existir depois, quando você escrever `thoric.nome`.

A linha inteira significa: "pega o valor que chegou no parâmetro `nome`, e guarda ele no campo `nome` deste objeto". Repare que os nomes não *precisam* ser iguais — funcionaria do mesmo jeito se o parâmetro se chamasse `n`:

```
def __init__(self, n, v, a):
    self.nome = n
    self.vida = v
    self.ataque = a
```

Isso funciona exatamente igual. Só é mais confuso de ler — por isso, quase todo mundo usa o mesmo nome dos dois lados. É uma convenção pra facilitar a leitura, não uma regra do Python.

4. O que acontece por dentro, passo a passo

Quando você escreve esta linha:

```
thoric = Personagem("Thoric", 100, 15)
```

o Python faz, na ordem, exatamente isto:

- **1.** Cria um objeto novo, vazio, do tipo `Personagem` — ainda sem nenhum atributo preenchido.
- **2.** Chama o método `__init__` desse objeto sozinho, automaticamente. Você não chama `__init__` na mão — ele já roda como parte de criar o objeto.
- **3.** Passa esse objeto recém-criado como `self`, e os valores `"Thoric", 100, 15` como `nome, vida, ataque`.
- **4.** Dentro do `__init__`, cada linha `self.algo = algo` grava um valor dentro *desse* objeto específico.
- **5.** Quando o `__init__` termina, o objeto já está completo — e é isso que fica guardado na variável `thoric`.

Ou seja: `Personagem("Thoric", 100, 15)` e as quatro linhas do Dia 1 (`thoric = Personagem()` + três atribuições) fazem exatamente a mesma coisa no final. A diferença é que o `__init__` faz tudo isso em uma linha, de forma automática, toda vez.

5. self em profundidade

`self` não é uma palavra reservada do Python, tecnicamente — poderia se chamar qualquer coisa. Mas *todo mundo* chama de `self`, em todo código Python do mundo, e usar outro nome só vai confundir quem lê depois (incluindo você mesmo, duas semanas depois). Então: sempre `self`, sem exceção.

O que `self` resolve é simples de esquecer porque parece óbvio demais: quando você tem *vários* objetos da mesma classe, como o Python sabe em qual objeto guardar cada valor?

```
thoric = Personagem("Thoric", 100, 15)
elara = Personagem("Elara", 60, 25)
```

Aqui, `__init__` roda *duas* vezes — uma pra cada linha. Na primeira vez, `self` é o objeto que vai virar `thoric`. Na segunda vez, `self` é um objeto completamente diferente, que vai virar `elara`. Cada chamada de `self.nome = nome` está escrevendo no objeto certo, porque `self` muda de acordo com qual objeto está sendo criado naquele momento. É por isso que `thoric.vida` e `elara.vida` podem ter valores diferentes mesmo vindo do mesmo `__init__`.

Pensando de outro jeito: `self` é a resposta à pergunta "de quem é esse atributo?" — sempre que você vê `self.algo` dentro de um método, leia como "o algo deste objeto aqui, não de nenhum outro".

6. Erros comuns (e como consertar)

Esses são os erros que praticamente todo mundo comete na primeira semana com `__init__`. Se um deles acontecer com vocês, é esperado — não é sinal de que algo está errado com o seu raciocínio.

Esquecer o self no def __init__

CÓDIGO COM ERRO

```
class Personagem:
    def __init__(nome, vida, ataque):
        self.nome = nome
```

O QUE O PYTHON MOSTRA

```
TypeError: __init__() takes 3 positional arguments but 4 were given
```

CORREÇÃO

```
def __init__(self, nome, vida, ataque):
```

O Python sempre manda o objeto como primeiro argumento, de forma automática — mesmo que você não peça. Se o primeiro parâmetro não se chama `self`, o objeto acaba caindo no lugar de `nome`, e sobra um argumento a mais. `self` precisa estar lá, sempre, em primeiro lugar.

Esquecer de passar os valores na hora de criar o objeto

CÓDIGO COM ERRO

```
thoric = Personagem()
```

O QUE O PYTHON MOSTRA

```
TypeError: __init__() missing 3 required positional
arguments: 'nome', 'vida', and 'ataque'
```

CORREÇÃO

```
thoric = Personagem("Thoric", 100, 15)
```

Se o `__init__` pede três parâmetros além do `self`, os três precisam ser fornecidos toda vez que a classe é chamada — não tem valor "padrão" a menos que você defina um (isso é um assunto pra mais pra frente).

Usar nome em vez de self.nome dentro de outro método

CÓDIGO COM ERRO

```
def mostrar_status(self):
    print(nome, vida)
```

O QUE O PYTHON MOSTRA

```
NameError: name 'nome' is not defined
```

CORREÇÃO

```
def mostrar_status(self):
    print(self.nome, self.vida)
```

`nome` e `vida` sozinhos só existiram como parâmetros *dentro* do `__init__`, no momento da criação. Depois disso, o único jeito de acessar esses valores é através do objeto: `self.nome`, dentro de um método, ou `thoric.nome`, fora da classe.

Achar que precisa chamar o __init__ na mão

CÓDIGO DESNECESSÁRIO

```
thoric = Personagem()
thoric.__init__("Thoric", 100, 15)
```

O JEITO CERTO

```
thoric = Personagem("Thoric", 100, 15)
```

O código acima até funciona, mas está fazendo o trabalho manualmente que o Python já faz sozinho. Nunca é preciso chamar `__init__` diretamente — ele roda automaticamente assim que a classe é chamada como se fosse uma função: `Personagem(...)`.

7. Perguntas frequentes

Por que tem dois underlines antes e depois de "init"?

Esse formato (`__nome__`) é reservado pelo próprio Python pra métodos especiais, que têm um comportamento automático embutido na linguagem. `__init__` é o mais comum, mas existem outros (você vão ver mais alguns futuramente). O apelido pra esse formato é "dunder" — de *double underscore*.

Preciso sempre nomear os parâmetros igual aos atributos?

Não, como mostrado na seção 3 — mas é fortemente recomendado, porque deixa o código mais fácil de ler. Usar nomes diferentes só pra ser diferente vai confundir mais do que ajudar.

Posso criar atributos que não vêm do __init__?

Sim — qualquer método pode fazer `self.algo = valor`, não só o `__init__`. O `__init__` é só o lugar mais comum e mais organizado pra criar os atributos principais, porque roda uma vez, logo no início da vida do objeto.

Toda classe precisa ter um __init__?

Não — no Dia 1, a classe `Personagem` funcionou sem nenhum construtor, só com atributos preenchidos na mão. Mas qualquer classe que tenha atributos que variam de objeto pra objeto praticamente sempre vai se beneficiar de ter um.

8. Dia 1 vs Dia 2, lado a lado

Dois formas diferentes de chegar exatamente no mesmo resultado — mesmo `thoric`, mesmos valores:

Dia 1 — sem construtor	Dia 2 — com <code>__init__</code>
<pre>class Personagem: nome = "" vida = 0 ataque = 0 thoric = Personagem() thoric.nome = "Thoric" thoric.vida = 100 thoric.ataque = 15</pre>	<pre>class Personagem: def __init__(self, nome, vida, ataque): self.nome = nome self.vida = vida self.ataque = ataque thoric = Personagem("Thoric", 100, 15)</pre>

Os dois criam um `thoric` idêntico. A diferença é só o caminho: quatro linhas repetidas toda vez, contra uma linha só, sempre que o construtor já existe.

9. Checklist mental

Antes de considerar um `__init__` pronto, confira:

- Começa com `def __init__(self, ...):` — com `self` em primeiro lugar?
- Todo parâmetro que deveria virar atributo tem uma linha `self.algo = algo` correspondente?
- Do lado esquerdo de cada atribuição está sempre `self.algo`, nunca só `algo`?
- Na hora de criar o objeto, todos os valores estão sendo passados, na ordem certa: `Personagem(valor1, valor2, valor3)`?
- Em nenhum lugar o `__init__` está sendo chamado manualmente com `.__init__(...)`?

10. Glossário rápido

construtor

método especial que roda automaticamente quando um objeto é criado; em Python, sempre chamado `__init__`.

self

primeiro parâmetro de qualquer método de uma classe; representa o próprio objeto sendo criado ou usado.

parâmetro

valor que uma função ou método recebe de fora, definido entre os parênteses do `def`.

atributo

valor guardado dentro de um objeto, acessado como `objeto.atributo`.

instância

outro nome pra "objeto criado a partir de uma classe" — `thoric` é uma instância de `Personagem`.

dunder

apelido informal pra nomes com dois underlines de cada lado (`__init__`, `__str__`, etc.), reservados pelo Python.

Se algo aqui ainda não fez sentido depois de ler duas vezes, tudo bem — é o tipo de coisa que costuma clicar de verdade só depois de escrever alguns `__init__` com a própria mão. Guarda a dúvida e traz pra aula.